

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Sistemas Operativos



Grupo 1

Ing. Angel Isidro Mercado

Tarea 5. Particionamiento de discos

320304435 - Cabrera Rojas Oscar

Ciudad Universitaria, CDMX a 23 de octubre del 2026

Índice

1. Particionamiento y formateo de discos **1**
1.1. Naturaleza y fundamentos del particionamiento 1
1.2. Mecanismos y niveles de formateo de disco 1

2. Esquema Master Boot Record (MBR) **3**
2.1. Estructura interna del sector de arranque maestro 3
2.2. Desglose de la entrada de partición y tipología 4
2.3. Limitaciones técnicas y direccionamiento CHS frente a LBA 5

3. Tabla de Particiones GUID (GPT) **6**
3.1. Arquitectura del estándar GPT y disposición de bloques lógicos 7
3.2. Identificadores únicos globales y estructura de la entrada de partición 8
3.3. Capacidades volumétricas, tolerancia a fallos y el estándar UEFI 9

4. Gestores de arranque **10**
4.1. Definición y ciclo de vida de inicialización 11
4.2. Arquitectura de carga multi-etapa 11
4.3. Carga en cadena y especificación Multiboot 12

5. Linux Loader (LILO) **13**
5.1. Arquitectura basada en mapas de sectores físicos 13
5.2. Rigidez operativa, barreras de geometría y obsolescencia 14

6. Grand Unified Bootloader (GRUB y GRUB 2) **14**
6.1. Evolución y arquitectura modular: de GRUB Legacy a GRUB 2 15
6.2. Reconocimiento dinámico de sistemas de archivos 16
6.3. Proceso de carga, entorno de rescate y configuración declarativa 16

7. Discos básicos y discos dinámicos **17**
7.1. Concepto y arquitectura del Administrador de Discos Lógicos 18
7.2. Tipología y modos de operación de volúmenes dinámicos 18
7.3. Diferencias estructurales y comparativa técnica 18

8. Creación y conversión de discos dinámicos **19**
8.1. Procedimientos administrativos de inicialización y conversión 19
8.2. El requisito de reserva del último megabyte (base de datos LDM) 20
8.3. Asimetría de reversibilidad y consideraciones operativas 20

9. Ventajas, limitaciones y estado tecnológico **20**
9.1. Ventajas operativas y de resiliencia 20
9.2. Desventajas, riesgos operativos e incompatibilidad 21
9.3. Obsolescencia y sustitución por Espacios de Almacenamiento (Storage Spaces) 21

10. Conclusiones personales **21**

Referencias **23**

1. Particionamiento y formateo de discos

El almacenamiento secundario en los sistemas de computación modernos constituye el nivel de persistencia fundamental sobre el cual se estructuran los sistemas de archivos y se preserva el estado del sistema operativo. A diferencia de la memoria principal, cuya volatilidad y limitaciones de direccionamiento físico restringen su uso a la ejecución transitoria de procesos, los dispositivos de almacenamiento masivo —tales como unidades de disco magnético y unidades de estado sólido— proporcionan almacenamiento no volátil estructurado como una secuencia unidimensional de bloques lógicos direccionables (*Logical Block Addressing*, LBA) (Silberschatz et al., 2006, §12.1, p. 408). No obstante, la utilización directa de un medio de almacenamiento físico en su totalidad sin una demarcación estructural previa resulta ineficiente e introduce riesgos severos de integridad, contención y compatibilidad. Por ello, la preparación de un soporte de almacenamiento para su aprovechamiento por parte del núcleo (*kernel*) demanda dos fases secuenciales e interdependientes: el particionamiento del medio y el formateo de las estructuras resultantes.

1.1. Naturaleza y fundamentos del particionamiento

El particionamiento de disco consiste en la división lógica de la capacidad total de un dispositivo de almacenamiento secundario en una o más regiones contiguas e independientes de bloques lógicos, de modo que cada región delimitada pueda ser tratada por el subsistema de almacenamiento del sistema operativo como si fuera una unidad física separada (Silberschatz et al., 2006, §12.5.1, p. 418). Mediante este procedimiento, el controlador del sistema de archivos no interactúa directamente con los límites globales del hardware subyacente, sino con abstracciones de bloques gestionadas por la capa de controladores de bloque del núcleo.

Desde una perspectiva arquitectónica, el particionamiento responde a requerimientos estructurales y operativos esenciales: previene el colapso del sistema aislando directorios volátiles (`/var` o `/tmp`) del sistema de archivos raíz (`/`) (Silberschatz et al., 2006, §11.2, p. 345); facilita esquemas multiarranque (*multi-boot*) y la aplicación de opciones de montaje restrictivas (`ro`, `noexec`, `nosuid`); y optimiza el rendimiento del área de intercambio (*swap*) y de particiones en bruto (*raw disk*) para motores de bases de datos relacionales (SGBD), eludiendo el búfer intermedio del núcleo (*buffer cache*) (Silberschatz et al., 2006, §12.5.1, p. 419).

1.2. Mecanismos y niveles de formateo de disco

Un dispositivo magnético o de estado sólido recién fabricado constituye un soporte carente de toda organización lógica de referencia. Para que el procesador y el sistema operativo puedan ubicar, almacenar y recuperar bloques de información de manera determinista, es imperativo establecer marcos de referencia tanto a nivel físico como a nivel de abstracción de datos. Este proceso de estructuración se clasifica rigurosamente en dos etapas: el formateo a bajo nivel y el formateo lógico (Silberschatz et al., 2006, §12.5.1, p. 418).

1.2.1. Formateo a bajo nivel (formateo físico)

El formateo a bajo nivel, también denominado formateo físico, es la operación mediante la cual se divide la superficie física del medio de grabación magnético o las matrices de

memoria en sectores de tamaño uniforme que la controladora de disco pueda leer y escribir de forma autónoma (Silberschatz et al., 2006, §12.5.1, p. 418). En los discos contemporáneos, este proceso es ejecutado exclusivamente en la fábrica durante el ciclo de producción, empleando servoposicionamiento de alta precisión para definir las pistas y sectores.

Cada sector físico generado se estructura a través de tres campos estandarizados: la *cabecera o preámbulo* (*preamble*), que provee sincronización de reloj y almacena las coordenadas físicas del sector (o identificador LBA); el *área de carga útil* (*payload*), que resguarda los datos del usuario (512 bytes tradicionales o 4096 bytes en formato avanzado 4Kn/512e); y la *cola con código de corrección de errores* (*Error-Correcting Code*, ECC), empleada por la controladora para verificar la integridad matemática del bloque y subsanar fallos blandos de lectura de forma autónoma (Silberschatz et al., 2006, §12.5.1, p. 418).

Asimismo, el formateo físico inicializa los mecanismos de gestión de defectos de la controladora. Se genera la lista primaria de defectos de fábrica (*Primary Defect List*, P-list), la cual omite de manera permanente los sectores imperfectos detectados durante las pruebas térmicas y magnéticas de manufactura. Durante la vida útil del disco, los sectores que degradan su relación señal a ruido son detectados por fallos ECC reiterados y se reasignan a sectores de reserva en las pistas de repuesto (*spare sectors*), registrándose en la lista creciente de defectos (*Grown Defect List*, G-list), todo ello de forma transparente para el sistema operativo (Silberschatz et al., 2006, §12.5.3, pp. 420–421).

1.2.2. Formateo lógico (creación del sistema de archivos)

Una vez que el medio ha sido formateado físicamente y subdividido en particiones mediante la tabla de particiones correspondiente, el sistema operativo no puede escribir archivos convencionales de inmediato. Se requiere el segundo paso, conocido como formateo lógico o creación del sistema de archivos (*high-level formatting*) (Silberschatz et al., 2006, §12.5.1, p. 418).

Durante el formateo lógico, el software del sistema operativo (por ejemplo, utilidades de usuario como `mkfs.ext4`, `mkfs.xfs` o `format.com`) escribe las estructuras de datos que modelan el sistema de archivos sobre la partición seleccionada. Dichas estructuras incluyen: superbloques o descriptores de grupo que definen la geometría y parámetros de montaje (Carretero Pérez et al., 2001, §8.2, pp. 448–450); tablas de inodos o tablas maestras de archivos (*Master File Table*, MFT) que almacenan atributos, permisos, marcas de tiempo y apuntadores a bloques de datos (Silberschatz et al., 2006, §11.1, p. 369); mapas de bits de espacio libre o cadenas de asignación (tablas FAT) para el seguimiento instantáneo de los bloques disponibles (Stallings, 2018, §11.5, pp. 524–526); y el directorio raíz inicial vacío.

Para incrementar la eficiencia en las transferencias de entrada/salida (E/S), los sistemas de archivos agrupan varios sectores lógicos en una unidad de asignación denominada *cluster* o bloque del sistema de archivos (frecuentemente configurado en 4096 bytes). Mientras que la controladora de disco transfiere datos a nivel de sector, el sistema operativo transfiere datos por bloques de asignación mediante acceso directo a memoria (*Direct Memory Access*, DMA), lo que reduce la fragmentación externa de los metadatos y promueve accesos contiguos en disco (Silberschatz et al., 2006, §12.5.1, p. 419).

La Tabla 1 resume las divergencias estructurales, operativas y de responsabilidad arquitectónica entre el formateo físico y el formateo lógico.

Tabla 1: Comparativa técnica entre el formateo a bajo nivel y el formateo lógico.

Criterio técnico	Formateo a bajo nivel (físico)	Formateo lógico (alto nivel)
Nivel de ejecución	Capa de hardware y microcódigo interno de la controladora de disco durante la manufactura.	Capa del sistema operativo mediante utilidades de espacio de usuario y controladores del kernel.
Objetivo estructural	Creación física de pistas, delimitación de sectores, encabezados de sincronización y campos ECC.	Creación de estructuras lógicas de indexación: superbloques, tablas de inodos, tablas de asignación y directorio raíz.
Unidad mínima de trabajo	Sector físico (típicamente 512 bytes o 4096 bytes en formato avanzado 4Kn).	Bloque del sistema de archivos o *cluster* (agrupación de sectores, típicamente de 4096 bytes).
Gestión de defectos	Identificación de sectores inservibles en fábrica (P-list) y sustitución dinámica por sectores de reserva (G-list).	Registro de bloques lógicos dañados dentro de los metadatos del sistema de archivos para evitar su asignación.
Dependencia de software	Estrictamente agnóstico al sistema operativo y al sistema de archivos a instalar.	Directamente dependiente del sistema operativo anfitrión y del sistema de archivos seleccionado (ext4, NTFS, XFS).
Impacto sobre los datos	Destrucción completa e irreversible de cualquier información y marcas magnéticas precedentes.	Inicialización o sobreescritura de las tablas de metadatos; los datos de bloques de usuario suelen quedar marcados como libres.

2. Esquema Master Boot Record (MBR)

El registro de arranque maestro (*Master Boot Record*, MBR) constituye el esquema clásico de particionamiento y carga de arranque introducido en 1983 con IBM PC DOS 2.0, consolidándose como el estándar de facto para la arquitectura x86 y los sistemas compatibles con IBM PC durante más de tres décadas (Silberschatz et al., 2006, §12.5.2, p. 419). El MBR reside invariablemente en el primer sector direccionable de un disco físico, correspondiente al sector lógico cero (LBA 0) o, bajo la nomenclatura geométrica clásica, al cilindro 0, cabeza 0 y sector 1. Con una longitud estricta de 512 bytes, este sector alberga tanto el código de máquina inicial para la secuencia de arranque como la tabla de particiones fundamental del medio (Unified Extensible Firmware Interface Forum, 2024, §5.2.1, p. 110).

2.1. Estructura interna del sector de arranque maestro

La arquitectura del sector MBR está dividida de forma rígida en tres áreas operativas no solapadas, cuya disposición responde a los límites de memoria y capacidades de direccionamiento del sistema básico de entrada/salida (*Basic Input/Output System*, BIOS) en modo real de 16 bits. La especificación formal de la interfaz de firmware extensible unificada (*Unified Extensible Firmware Interface*, UEFI) documenta la distribución binaria de este sector como se detalla en la Tabla 2 (Unified Extensible Firmware Interface Forum, 2024, §5.2.1,

Tabla 2: Estructura general del sector Master Boot Record (512 bytes en LBA 0).

Desplazamiento	Tamaño	Descripción y función en el sistema
0x000 – 0x1BD	446 bytes	Código ejecutable inicial de arranque (*bootstrap code*).
0x1BE – 0x1CD	16 bytes	Registro de la partición primaria 1.
0x1CE – 0x1DD	16 bytes	Registro de la partición primaria 2.
0x1DE – 0x1ED	16 bytes	Registro de la partición primaria 3.
0x1EE – 0x1FD	16 bytes	Registro de la partición primaria 4.
0x1FE – 0x1FF	2 bytes	Firma de arranque mágica (*boot signature*): 0x55AA.

p. 110).

El flujo de inicialización depende de la coordinación de estas tres regiones. Tras las comprobaciones del encendido (*Power-On Self-Test*, POST), el firmware BIOS transfiere los 512 bytes del MBR a la dirección de memoria física 0x7C00 y cede la ejecución al procesador. El código del bootstrap inicializa registros, examina las cuatro entradas de 16 bytes de la tabla de particiones (desplazamientos 0x1BE a 0x1FD) para ubicar la partición marcada como activa, y transfiere el control a su sector inicial, denominado registro de arranque del volumen (*Volume Boot Record*, VBR) (Silberschatz et al., 2006, §12.5.2, p. 420). En sistemas como Windows, los desplazamientos 0x1B8 a 0x1BD albergan la firma de disco (*Disk Signature*) de 32 bits para indexación del kernel.

Por último, los desplazamientos 0x1FE y 0x1FF contienen la firma mágica 0x55AA (0xAA55 en *little-endian*), requerida obligatoriamente por el BIOS antes de ejecutar el código; su ausencia provoca la detención del arranque con advertencias de disco no arrancable.

2.2. Desglose de la entrada de partición y tipología

Cada una de las cuatro entradas de partición de 16 bytes en la tabla MBR modela de forma autónoma los límites, atributos y naturaleza del sistema de archivos albergado en la partición. La Tabla 3 expone la composición canónica de este registro según el estándar UEFI (Unified Extensible Firmware Interface Forum, 2024, §5.2.1, p. 111).

Debido a que el tamaño total de la tabla en el MBR está acotado a 64 bytes, solo es factible definir un máximo de cuatro **particiones primarias**. Una partición primaria es una región contigua direccionada directamente por una de las cuatro entradas del MBR y capaz de albergar un sistema operativo con su respectivo sector de arranque.

Para superar la restricción de cuatro particiones, la industria introdujo el concepto de **partición extendida** (identificada históricamente por los códigos 0x05 o 0x0F). Una partición extendida no almacena datos de usuario directamente; actúa como un contenedor secundario que subdivide su espacio asignado en una cantidad arbitraria de **unidades lógicas**.

La arquitectura de las unidades lógicas no se apoya en una tabla centralizada, sino en una estructura encadenada de registros conocida como registro de arranque extendido (*Extended Boot Record*, EBR). Cada partición lógica va precedida inmediatamente por un sector EBR que replica parcialmente la estructura del MBR: una primera entrada de 16 bytes que describe los límites físicos (LBA inicial y tamaño en sectores) de la unidad lógica asociada, una segunda entrada de 16 bytes que funciona como apuntador hacia el siguiente

Tabla 3: Estructura interna de 16 bytes de la entrada de partición MBR.

Campo binario	Tamaño	Definición técnica y valores operacionales
`BootIndicator`	1 byte	Indicador de partición activa: `0x80` marca partición arrancable; `0x00` partición inactiva. Otros valores son considerados inválidos por el BIOS.
`StartCHS`	3 bytes	Coordenadas geométricas de inicio bajo esquema Cilindro-Cabeza-Sector: byte 0 cabeza, byte 1 sector (bits 0–5) y cilindro alto (bits 6–7), byte 2 cilindro bajo.
`OSType`	1 byte	Identificador de tipo de sistema de archivos o sistema operativo (*Partition ID*): `0x07` (NTFS/exFAT), `0x83` (Linux nativo), `0x82` (Linux swap), `0x05`/`0x0F` (partición extendida), `0x0C` (FAT32 LBA).
`EndCHS`	3 bytes	Coordenadas geométricas de finalización bajo esquema Cilindro-Cabeza-Sector (sujetas a los mismos límites de codificación que `StartCHS`).
`StartingLBA`	4 bytes	Dirección de bloque lógico (LBA) del primer sector de la partición (entero sin signo de 32 bits, codificado en *little-endian*).
`SizeInLBA`	4 bytes	Número total de sectores contenidos en la partición (entero sin signo de 32 bits, codificado en *little-endian*).

EBR de la cadena, dos entradas nulas y la firma de arranque `0x55AA`. Si bien este encadenamiento permite crear decenas de unidades lógicas, presenta una fragilidad crítica: la corrupción o pérdida de un único sector EBR en la secuencia rompe la lista e inutiliza de inmediato todas las particiones posteriores.

2.3. Limitaciones técnicas y direccionamiento CHS frente a LBA

A pesar de su trascendencia histórica, el esquema MBR presenta severas restricciones arquitectónicas que precipitaron su obsolescencia ante el advenimiento de medios de almacenamiento de alta densidad y arquitecturas de 64 bits:

2.3.1. La barrera de los 2.2 Terabytes en direccionamiento de 32 bits

La limitación estructural más crítica del MBR radica en el tamaño de los campos destinados a registrar las direcciones de bloques lógicos en cada entrada de la tabla: tanto el campo de desplazamiento inicial (`StartingLBA`) como el de volumen de sectores (`SizeInLBA`) poseen una longitud fija de 4 bytes (32 bits sin signo) (Unified Extensible Firmware Interface Forum, 2024, §5.2.1, p. 111). Con $2^{32} = 4{,}294{,}967{,}296$ sectores indexables y un tamaño estándar de sector de 512 bytes de carga útil, la capacidad teórica absoluta direccionable asciende a:

$$C_{\text{máx}} = 2^{32} \text{ sectores} \times 512 \text{ bytes} = 2{,}199{,}023{,}255{,}552 \text{ bytes} = 2.0 \text{ TiB} \approx 2.2 \text{ TB} \quad (1)$$

En consecuencia, cuando se conecta una unidad cuya capacidad física excede los 2.2 TB a un sistema gestionado mediante MBR con sectores de 512 bytes, todo el espacio de

almacenamiento que sobrepase el límite de los 2^{32} bloques resulta matemáticamente inaccesible para el controlador del disco, quedando desaprovechado. Aunque el estándar de formato avanzado con emulación de sectores mayores (4Kn) mitiga parcialmente este techo al multiplicar la base por ocho, no resuelve las deficiencias de redundancia e integridad del protocolo.

2.3.2. El cuello de botella del direccionamiento geométrico CHS

En los albores de la arquitectura PC, la comunicación con el subsistema de disco se realizaba mediante coordenadas físicas de cilindro, cabeza y sector (CHS) a través de la interrupción BIOS `INT 13h`. Los campos CHS en la entrada de partición MBR asignaban 10 bits al cilindro (valores de 0 a 1023), 8 bits a la cabeza (valores de 0 a 255) y 6 bits al sector (valores de 1 a 63) (Unified Extensible Firmware Interface Forum, 2024, §5.2.1, p. 111).

Bajo esta parametrización, el rango máximo direccionable por hardware mediante la interfaz clásica del BIOS estaba restringido a:

$$1024 \text{ cilindros} \times 256 \text{ cabezas} \times 63 \text{ sectores} \times 512 \text{ bytes} = 8{,}455{,}716{,}864 \text{ bytes} \approx 8.46 \text{ GB} \quad (2)$$

Cualquier disco cuya capacidad sobrepasara los 8.4 GB requirió la formulación de extensiones de la interrupción BIOS `INT 13h` y la transición al direccionamiento lineal por bloques lógicos (LBA), relegando los campos CHS a valores de relleno (`0xFEFFFF`) que señalan la inutilidad de las coordenadas geométricas en medios modernos.

2.3.3. Ausencia total de redundancia y vulnerabilidad ante fallos

El diseño del MBR almacena la tabla de particiones en una única ubicación física fija: el primer sector de 512 bytes en LBA 0. El estándar no contempla ninguna copia de respaldo en otro extremo del disco, ni implementa sumas de verificación o códigos de redundancia cíclica (*Cyclic Redundancy Check*, CRC) para certificar la integridad de los metadatos.

Esta carencia de tolerancia a fallos implica que cualquier daño en el sector LBA 0 —sea originado por un desgaste electromagnético de la superficie, un corte de energía durante una reescritura, o la acción de código malicioso especializado en el sector de arranque (*bootkits*)— destruye de manera irreversible el mapa de particiones del disco, imposibilitando que el sistema operativo localice los límites de sus volúmenes sin herramientas forenses de reconstrucción heurística.

3. Tabla de Particiones GUID (GPT)

La tabla de particiones GUID (*GUID Partition Table*, GPT) es el estándar contemporáneo de organización y particionamiento de medios de almacenamiento secundario, formulado inicialmente por Intel como parte de la especificación de la interfaz de firmware extensible (*Extensible Firmware Interface*, EFI) y consolidado como elemento cardinal del estándar UEFI (Unified Extensible Firmware Interface Forum, 2024, §5.1, p. 110). Diseñada para subsanar íntegramente las limitaciones físicas y lógicas del registro de arranque maestro (MBR), la arquitectura GPT abandona las restricciones del direccionamiento de 32 bits, la dependencia de sectores ciegos de código máquina y la ausencia de mecanismos de verificación de integridad, estableciendo un esquema fundamentado en direccionamiento por bloques lógicos (LBA) de 64 bits, identificadores únicos universales y redundancia estructural simétrica en ambos extremos del disco.

Tabla 4: Disposición arquitectónica del espacio de bloques lógicos en un disco GPT.

Ubicación de bloques	Estructura	Función y características en la plataforma
LBA 0	MBR de protección	Estructura MBR de compatibilidad que declara una única partición de tipo `0xEE` para proteger el medio ante herramientas legadas.
LBA 1	Cabecera primaria GPT	Metadatos maestros del disco: firmas, apuntadores LBA, sumas de verificación CRC32 y número de entradas de partición.
LBA 2 – LBA 33	Matriz primaria de entradas	Matriz de descriptores de partición (128 entradas canónicas de 128 bytes cada una, totalizando 16 384 bytes o 32 sectores).
LBA 34 – LBA $N-34$	Área de datos de usuario	Espacio físico continuo disponible para la creación de particiones de sistemas de archivos y áreas operativas del sistema operativo.
LBA $N-33$ – LBA $N-2$	Matriz de respaldo	Réplica bit a bit de la matriz de entradas de partición ubicada al final del dispositivo para recuperación ante corrupción.
LBA $N-1$	Cabecera de respaldo GPT	Copia de respaldo de la cabecera primaria situada en el último sector físico del disco (N corresponde al número total de sectores).

3.1. Arquitectura del estándar GPT y disposición de bloques lógicos

A diferencia de la concentración monolítica de metadatos característica del MBR en un único sector de 512 bytes, la especificación GPT distribuye sus estructuras a lo largo de un conjunto predeterminado de bloques lógicos contiguos al inicio y al final del medio físico (Unified Extensible Firmware Interface Forum, 2024, §5.3.1, pp. 113–115). Esta topología garantiza tanto la interoperabilidad retrospectiva como la tolerancia ante fallos electromagnéticos en los sectores de cabecera. La Tabla 4 resume la distribución espacial de los bloques lógicos en un medio formateado bajo GPT.

La articulación de estos segmentos obedece a un diseño de coexistencia y alta resiliencia. En el bloque LBA 0, el MBR de protección (*Protective MBR*) define una partición de tipo `0xEE` que abarca la totalidad del disco (o hasta el límite de $2^{32}-1$ sectores), previniendo que herramientas legadas de BIOS asuman erróneamente que el medio carece de formato y sobreescriban sus datos de forma destructiva (Unified Extensible Firmware Interface Forum, 2024, §5.2.2, p. 112).

A continuación, la cabecera primaria en LBA 1 (de 92 bytes) almacena los parámetros cardinales: la firma ASCII "`EFI PART`" (`0x5452415020494645` en *little-endian*), las sumas de comprobación CRC32 de la cabecera y de la matriz, el identificador único global (*Disk GUID*) y los punteros a los límites utilizables (*First Usable LBA* en LBA 34 y *Last Usable LBA* en LBA $N-34$) y a la cabecera alterna en LBA $N-1$ (Unified Extensible Firmware Interface Forum, 2024, §5.3.1, pp. 113–115). Los bloques LBA 2 a 33 reservan 16 384 bytes para 128 entradas de partición independientes de 128 bytes cada una, erradicando la dependencia de particiones extendidas y EBR. Finalmente, los sectores LBA $N-33$ a $N-1$ alojan una réplica simétrica de la matriz y de la cabecera, garantizando la recuperación automática

Tabla 5: Estructura interna de 128 bytes de una entrada de partición GPT.

Desplazamiento	Tamaño	Campo técnico y significado funcional
`0x00` – `0x0F`	16 bytes	Tipo de partición GUID (*Partition Type GUID*): Identifica la finalidad operativa de la partición a nivel de sistema.
`0x10` – `0x1F`	16 bytes	GUID único de la partición (*Unique Partition GUID*): Identificador generado de forma unívoca para la instancia específica.
`0x20` – `0x27`	8 bytes	Bloque lógico inicial (*Starting LBA*): Entero sin signo de 64 bits en orden *little-endian*.
`0x28` – `0x2F`	8 bytes	Bloque lógico final (*Ending LBA*): Entero sin signo de 64 bits que marca el último bloque ocupado (inclusive).
`0x30` – `0x37`	8 bytes	Banderas de atributos (*Attribute Flags*): Registro de 64 bits para control de montaje, privilegios de solo lectura y visibilidad.
`0x38` – `0x7F`	72 bytes	Nombre legible de la partición (*Partition Name*): Cadena alfanumérica de hasta 36 caracteres codificada en UTF-16LE.

ante corrupción de los primeros bloques.

3.2. Identificadores únicos globales y estructura de la entrada de partición

Cada partición definida en la matriz GPT está descrita por una estructura estandarizada de 128 bytes de longitud (Unified Extensible Firmware Interface Forum, 2024, §5.3.3, pp. 118–120). La Tabla 5 detalla la anatomía de este registro binario.

A diferencia de los códigos de un solo byte del MBR (donde colisionaban frecuentemente esquemas de distintos fabricantes), GPT emplea identificadores únicos universales (*Universally Unique Identifier*, UUID o GUID) de 128 bits (16 bytes), cuya representación formal sigue la convención hexadecimal estructurada de 32 dígitos separados por guiones: `XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX`.

El uso de los GUID en las entradas de partición comprende dos funciones diferenciadas:

- **Tipo de partición GUID (*Partition Type GUID*):** Define unívocamente la categoría y el rol que la partición cumple en la plataforma. Por ejemplo, la partición de sistema EFI (ESP) se identifica por el GUID canónico `C12A7328-F81F-11D2-BA4B-00A0C93EC93B`; la partición de datos básicos de Microsoft (NTFS, FAT32 o exFAT) utiliza `EBD0A0A2-B9E5-4433-87C0-68B6B72699C7`; las particiones genéricas de sistemas de archivos nativos de Linux (ext4, XFS, Btrfs) emplean `0FC636FD-8440-474E-99D6-0831656E4147`; y las áreas de intercambio swap de Linux se marcan con `0657FD6D-A4AB-43C4-84E5-0933C84B4F4F` (Unified Extensible Firmware Interface Forum, 2024, §5.3.3, p. 119).
- **GUID único de la partición (*Unique Partition GUID*):** Es un valor pseudoaleatorio único generado conforme al estándar RFC 4122 al momento de inicializar la partición. Este identificador es permanente e inmune a las modificaciones en la numeración física de las unidades (por ejemplo, cuando un disco cambia de canal SATA o se reordena en el bus PCIe). Los núcleos de sistemas operativos modernos emplean este identificador permanente (representado como `PARTUUID` en entornos GNU/Linux) dentro del archivo `/etc/fstab` para montar sistemas de archivos sin el riesgo de asignaciones

cruzadas derivado de nombres volátiles como `/dev/sda1` o `/dev/sdb1`.

Asimismo, los campos `StartingLBA` y `EndingLBA` de 64 bits permiten definir particiones individuales con límites espaciales masivos, mientras que los atributos de 64 bits proporcionan banderas estandarizadas a nivel de firmware: el bit 0 indica que la partición es indispensable para que el sistema funcione y no debe ser eliminada por el usuario; el bit 60 declara la partición como de solo lectura; el bit 62 la oculta de los exploradores gráficos convencionales; y el bit 63 prohíbe el montaje automático (*automount*) durante el arranque (Unified Extensible Firmware Interface Forum, 2024, §5.3.3, p. 120).

3.3. Capacidades volumétricas, tolerancia a fallos y el estándar UEFI

El diseño de GPT resuelve estructuralmente todos los cuellos de botella que afectaban a los sistemas de almacenamiento bajo MBR, proporcionando tres pilares arquitectónicos:

3.3.1. Capacidad volumétrica con direccionamiento de 64 bits

Al implementar campos de 64 bits (8 bytes) para registrar la ubicación y longitud de los bloques lógicos, GPT indexa hasta $2^{64} \approx 1.84 \times 10^{19}$ sectores. En medios con sectores convencionales de 512 bytes, la capacidad física direccionable absoluta asciende a:

$$C_{\text{máx}} = 2^{64} \times 512 \text{ bytes} = 9{,}444{,}732{,}965{,}739{,}290{,}427{,}392 \text{ bytes} = 8 \text{ ZiB} \approx 9.44 \text{ ZB} \quad (3)$$

La cota máxima de 8 Zettabytes (equivalente a ocho mil millones de Terabytes) erradica cualquier posibilidad práctica de obsolescencia volumétrica en el almacenamiento secundario previsible. En unidades de formato avanzado nativas (4Kn) con sectores de 4096 bytes, la capacidad direccionable alcanza los 64 ZiB.

3.3.2. Mecanismos de verificación de integridad y autorreparación mediante CRC32

La debilidad monopunto del MBR queda superada en GPT mediante el cálculo algorítmico obligatorio de redundancia cíclica (*Cyclic Redundancy Check*, CRC32) de 32 bits (Unified Extensible Firmware Interface Forum, 2024, §5.3.1, p. 114). La cabecera primaria almacena su propio CRC32 y el CRC32 de la matriz de particiones contigua; de forma simétrica, la cabecera de respaldo y la matriz secundaria preservan sus propios códigos de comprobación.

Durante la inicialización del sistema, el firmware UEFI calcula el CRC32 de las estructuras en disco y lo compara con los valores declarados en las cabeceras. Si se detecta una corrupción binaria en la cabecera o matriz primaria (debido a sectores defectuosos, escrituras abortadas o interferencia de software), el sistema rechaza los metadatos inconsistentes y restaura automáticamente las estructuras maestras a partir de la copia secundaria situada al final del medio físico, garantizando una alta resiliencia operativa.

3.3.3. La Partición de Sistema EFI (ESP) y el nuevo paradigma de arranque

En el modelo MBR tradicional, la fase de arranque dependía de la ejecución de código máquina en modo real de 16 bits insertado a ciegas en los 446 bytes del bootstrap o en sectores ocultos contiguos al MBR. En contraposición, el estándar UEFI prescribe la existencia

Tabla 6: Comparativa técnica integral entre los esquemas de particionamiento MBR y GPT.

Característica técnica	Master Boot Record (MBR)	GUID Partition Table (GPT)
Direccionamiento LBA	Campos rígidos de 32 bits en la tabla de particiones.	Campos nativos de 64 bits en todas las entradas de partición.
Capacidad máxima teórica	2.2 Terabytes (con sectores estándar de 512 bytes).	8 Zettabytes (512 bytes) o 64 Zettabytes (sectores 4Kn).
Límite de particiones	4 particiones primarias (o 3 primarias y 1 extendida con unidades lógicas encadenadas).	Mínimo 128 particiones independientes directas (sin particiones extendidas ni EBR).
Tolerancia a fallos y respaldo	Estructura monopunto en LBA 0; carece de respaldo y de sumas de integridad.	Copia simétrica de cabecera y matriz al final del disco; verificación obligatoria por CRC32.
Identificación de particiones	Códigos de tipo de 1 byte propensos a colisiones entre fabricantes.	GUID/UUID universales de 128 bits para el tipo de partición y para la instancia única.
Compatibilidad de arranque	Exclusivamente firmware BIOS heredado (*Legacy BIOS*).	Firmware UEFI contemporáneo (compatible con BIOS mediante MBR de protección).

de una partición dedicada: la partición de sistema EFI (*EFI System Partition*, ESP) (Unified Extensible Firmware Interface Forum, 2024, §13.3, pp. 530–532).

La ESP se formatea obligatoriamente bajo el sistema de archivos FAT (generalmente FAT32) y alberga binarios compilados en formato ejecutable portátil de 64 bits (PE32+). En lugar de saltar a sectores físicos del disco, el firmware UEFI navega directamente por la jerarquía de directorios de la ESP (como `\EFI\BOOT\BOOTX64.EFI`, `\EFI\Microsoft\Boot\bootmgfw.efi` o `\EFI\ubuntu\grubx64.efi`) y ejecuta los cargadores en modo nativo de 64 bits, independizando por completo la fase de arranque de la geometría del hardware subyacente.

La Tabla 6 condensa las diferencias fundamentales entre el esquema MBR clásico y la tabla de particiones GUID.

4. Gestores de arranque

El encendido de un sistema de computación inicia un estado transitorio en el cual la memoria de acceso aleatorio (RAM) se encuentra vacía o con contenido residual indefinido, y los registros internos del procesador apuntan a una dirección fija en memoria de solo lectura (ROM) donde reside el firmware básico del sistema (Silberschatz et al., 2006, §2.4, p. 64). Puesto que el firmware carece del diseño, controladores y políticas necesarias para gestionar los subsistemas de archivos avanzados, la concurrencia de procesos o la seguridad de usuarios, se requiere un intermediario de software especializado: el gestor o cargador de arranque (*bootloader* o *bootstrap loader*) (Silberschatz et al., 2006, §12.5.2, p. 419).

4.1. Definición y ciclo de vida de inicialización

Un gestor de arranque es un programa ejecutable de bajo nivel cuya finalidad cardinal consiste en preparar el hardware mínimo de la plataforma, localizar la imagen del núcleo (*kernel*) del sistema operativo en el medio de almacenamiento secundario, transferirla a la memoria principal junto con sus imágenes de soporte inicial, y cederle el control absoluto del procesador (Stallings, 2018, §2.8, pp. 80–82).

El ciclo de vida de inicialización articula cinco responsabilidades técnicas esenciales (Stallings, 2018, §2.8, pp. 80–82): conmutar la CPU desde el modo real de 16 bits heredado hacia el modo protegido de 32 bits o modo largo (*Long Mode*) de 64 bits para acceder sin trabas a la RAM física; consultar las tablas de particiones (MBR o GPT) e identificar la partición de arranque; transferir secuencialmente a memoria la imagen binaria del núcleo (`vmlinuz`) junto con el disco RAM inicial (*initial ramdisk*, *initrd* o *initramfs*) con sus controladores esenciales (Silberschatz et al., 2006, §12.5.2, p. 419); transmitir los parámetros de la línea de órdenes del kernel (*kernel command line*, tales como `root=PARTUUID=...` o `ro`); y saltar irrevocablemente al punto de entrada del núcleo (*kernel entry point*), cediendo el gobierno integral de la plataforma (Silberschatz et al., 2006, §2.4, p. 64).

La transición de control entre el firmware y el cargador difiere diametralmente entre plataformas. Bajo BIOS clásico, el firmware deposita ciegamente los 512 bytes del MBR en la dirección de memoria `0x7C00` y transfiere el flujo en modo real de 16 bits sin controladores protegidos. En contraste, el firmware UEFI inicializa los buses y controladores en modo nativo de 64 bits, proveyendo servicios de arranque (*Boot Services*) y protocolos de lectura de sistemas de archivos FAT para ejecutar aplicaciones PE32+ en la partición ESP (Unified Extensible Firmware Interface Forum, 2024, §2.1, pp. 24–26). El cargador utiliza dichos servicios hasta invocar la primitiva `ExitBootServices()`, momento en que el firmware desactiva sus estructuras transitorias y entrega la memoria física al sistema operativo.

4.2. Arquitectura de carga multi-etapa

La restricción histórica del sector de arranque maestro (MBR) a 512 bytes —de los cuales únicamente 446 bytes están disponibles para código máquina ejecutable— impuso una barrera insalvable para el software de inicialización. Resulta físicamente imposible empaquetar en 446 bytes la lógica de decodificación de sistemas de archivos avanzados (tales como ext4, XFS o Btrfs), el análisis de tablas de particiones dinámicas, la renderización de interfaces gráficas interactivas o el descifrado criptográfico de volúmenes (Silberschatz et al., 2006, §12.5.2, p. 419).

Para superar este confinamiento espacial, la ingeniería concibió la **arquitectura de carga multi-etapa** (*multi-stage bootloaders*). Como sintetiza la Tabla 7, la primera etapa (*Stage 1*) en los 446 bytes del MBR localiza y transfiere a memoria la etapa intermedia (*Stage 1.5* o imagen de núcleo *core image*) alojada en el espacio post-MBR (*MBR gap*); ésta incorpora controladores mínimos para acceder a la segunda etapa (*Stage 2*) en el directorio `/boot/`, la cual despliega la interfaz de usuario, procesa la configuración y transfiere el control al núcleo (Free Software Foundation, 2024a). En contraposición, las plataformas UEFI prescinden de esta fragmentación ejecutando directamente cargadores compilados en formato PE32+ desde la partición ESP.

La Tabla 7 sintetiza las etapas de esta arquitectura frente al modelo directo de UEFI.

Tabla 7: Fases de la arquitectura de carga de arranque multi-etapa frente a UEFI.

Etapa de carga	Ubicación física	Función y límites operativos
Stage 1	LBA 0 (MBR) o VBR (512 bytes)	Código en ensamblador de 16 bits; localiza y carga en memoria el Stage 1.5 mediante interrupciones BIOS básicas.
Stage 1.5 (*Core*)	Espacio post-MBR (*MBR gap*)	Controladores mínimos de lectura de sistemas de archivos; permite abrir archivos por su ruta lógica.
Stage 2	Sistema de archivos (`/boot/`)	Interfaz de selección interactiva, carga de configuración, descompresión del kernel y paso de parámetros.
Cargador UEFI	Partición ESP (archivo PE32+)	Ejecución directa en modo de 64 bits; prescinde de fases intermedias gracias a los servicios del firmware.

4.3. Carga en cadena y especificación Multiboot

En entornos multiarranque donde coexisten sistemas operativos con diferentes filosofías de inicialización (como GNU/Linux y Microsoft Windows), los gestores de arranque implementan dos mecanismos de cooperación arquitectónica:

4.3.1. Carga en cadena (*chainloading*)

La carga en cadena es una técnica mediante la cual el gestor de arranque primario no asume la tarea de cargar directamente el núcleo del sistema operativo de destino, sino que localiza el sector de arranque de la partición (VBR) o el binario UEFI del cargador secundario propio de ese sistema operativo, lo transfiere a memoria y le delega el control de forma transparente (Free Software Foundation, 2024a).

Este desacoplamiento resulta imprescindible cuando se requiere arrancar sistemas propietarios cuyo proceso de inicialización interno no está documentado o depende de estructuras propietarias complejas. Por ejemplo, cuando GRUB debe arrancar Microsoft Windows en una máquina con BIOS, no intenta interpretar los ejecutables del núcleo de Windows; en su lugar, aplica la orden `chainloader +1` sobre la partición donde reside el Administrador de Arranque de Windows (*Windows Boot Manager*, `bootmgr`), transfiriéndole el control para que el software nativo de Microsoft prosiga con su secuencia de carga. En entornos UEFI, el chainloading consiste simplemente en invocar el ejecutable de Windows (`\EFI\Microsoft\Boot\bootmgfw.efi`) mediante los servicios del firmware.

4.3.2. La especificación Multiboot

Con el propósito de erradicar la necesidad de que cada sistema operativo libre desarrolle su propio cargador de arranque incompatible con los demás, la Free Software Foundation formuló la **especificación Multiboot** (*Multiboot Specification*) (Free Software Foundation, 2024b).

Multiboot define un contrato formal y abierto entre el gestor de arranque y el núcleo del sistema operativo. Todo kernel diseñado conforme a esta norma contiene en sus primeros 8192 bytes una cabecera estandarizada (*Multiboot Header*) identificada por una constante mágica (`0x1BADB002`). Al detectar esta firma, el gestor de arranque garantiza al núcleo un estado homogéneo del procesador al momento del salto:

- La CPU se encuentra ya en modo protegido de 32 bits (o modo de 64 bits en Multiboot2), con la paginación desactivada y las interrupciones enmascaradas.
- El registro `EAX` contiene el número mágico de confirmación (`0x2BADB002`).
- El registro `EBX` almacena un puntero a una estructura de datos informativa en memoria (*Multiboot Information Structure*), la cual detalla el mapa de memoria RAM física del equipo, la ubicación de los módulos cargados (como el initramfs) y la línea de órdenes suministrada por el usuario.

Gracias a esta estandarización, un único gestor de arranque como GRUB puede cargar cualquier núcleo conforme a Multiboot sin modificaciones de código ni artificios de emulación de sectores.

5. Linux Loader (LILO)

Desarrollado en 1992 por Werner Almesberger, el cargador de Linux (*Linux Loader*, LILO) se erigió como el gestor de arranque predominante para sistemas operativos basados en GNU/Linux durante la década de 1990 y los primeros años del siglo XXI (Almesberger & Coffman, 2000). Diseñado en los albores del software libre para computadoras personales basadas en arquitectura x86, LILO proporcionó el primer mecanismo estable para arrancar núcleos Linux monolíticos comprimidos y gestionar configuraciones multiarranque sin depender de cargadores comerciales.

5.1. Arquitectura basada en mapas de sectores físicos

La característica medular que distingue a LILO de los cargadores modernos radica en su naturaleza estrictamente agnóstica al sistema de archivos (*filesystem-agnostic*) (Almesberger & Coffman, 2000). LILO no incorpora controladores ni bibliotecas para decodificar estructuras de sistemas de archivos en tiempo de ejecución; desconoce por completo la noción de superbloques, tablas de inodos, árboles B o estructuras de directorios.

Para solventar la localización del núcleo sin leer el sistema de archivos durante el encendido, LILO desacopla la resolución de rutas mediante dos fases: en la fase de espacio de usuario, el administrador edita `/etc/lilo.conf` y ejecuta la utilidad `/sbin/lilo`, la cual invoca llamadas `ioctl` (`FIBMAP` o `FIEMAP`) para traducir la ruta del kernel (`/boot/vmlinuz`) a una lista física de sectores LBA o CHS grabada en `/boot/map` e inyectada en el Stage 1 y Stage 2 (Silberschatz et al., 2006, §2.4, p. 64); en la fase de prearranque, el Stage 1 carga el Stage 2 y éste lee directamente los sectores brutos de `/boot/map` transfiriéndolos a memoria RAM mediante interrupciones BIOS antes de saltar al punto de entrada del núcleo (Almesberger & Coffman, 2000).

Esta arquitectura otorgó a LILO una huella de código diminuta, un consumo mínimo de recursos de memoria y una elevada velocidad de carga, al eludir la sobrecarga computacional de procesar estructuras jerárquicas en disco.

5.2. Rigidez operativa, barreras de geometría y obsolescencia

A pesar de su eficiencia histórica, la arquitectura basada en mapas estáticos de sectores introdujo graves vulnerabilidades operativas y limitaciones que precipitaron su obsolescencia tecnológica:

5.2.1. La regla obligatoria de reinstalación del mapa

Dado que LILO depende de la posición física exacta de los bloques en la superficie del disco, cualquier alteración en el medio invalida el mapa. Si se compila o actualiza el kernel, si se genera un nuevo initramfs, o si los bloques de `/boot` se desplazan debido a una reorganización de almacenamiento o desfragmentación, los sectores almacenados en `/boot/map` apuntarán a datos obsoletos o a espacio libre. Si el administrador olvida ejecutar la orden `/sbin/lilo` en el sistema operativo antes de reiniciar, LILO intentará cargar basura binaria a la memoria RAM, provocando un fallo catastrófico (*kernel panic*) o el bloqueo del equipo (Almesberger & Coffman, 2000).

5.2.2. La barrera geométrica de los 1024 cilindros del BIOS

En sus versiones tempranas, LILO operaba bajo la interfaz clásica de la interrupción BIOS `INT 13h` mediante coordenadas CHS. Debido a los 10 bits asignados al cilindro, LILO era incapaz de direccionar sectores más allá del cilindro 1023 (correspondiente a la cota de 8.4 GB o 504 MB en BIOS antiguos). Si los bloques físicos del kernel o de `/boot/map` se grababan más allá de este límite geométrico, el instalador abortaba con errores de geometría insalvables. Aunque revisiones posteriores incorporaron la directiva `lba32` para emplear extensiones de direccionamiento por bloques de 32 bits, el gestor permaneció condicionado por las limitaciones del firmware BIOS heredado.

5.2.3. Diagnóstico de fallos en el prompt de arranque

Una característica emblemática de LILO era su mecanismo de diagnóstico visual: imprimía las letras de su propio nombre conforme completaba con éxito cada fase de su secuencia de carga. La Tabla 8 detalla el significado de estos códigos de depuración (Almesberger & Coffman, 2000).

La carencia de una consola interactiva de rescate con línea de órdenes, la imposibilidad de explorar sistemas de archivos en caliente y la incapacidad estructural para gestionar volúmenes lógicos dinámicos (LVM), RAID por software, particiones GPT y firmware UEFI motivaron que la comunidad de desarrollo migrara definitivamente hacia cargadores avanzados con reconocimiento dinámico de sistemas de archivos, encabezados por GRUB.

6. Grand Unified Bootloader (GRUB y GRUB 2)

Concebido inicialmente en 1995 por Erich Boleyn para arrancar el micronúcleo GNU Mach en el sistema operativo GNU Hurd, y adoptado formalmente por el proyecto GNU de la Free Software Foundation (FSF), el Gran Cargador Unificado de Arranque (*Grand Unified Bootloader*, GRUB) representa el estándar contemporáneo de facto en gestores de arranque para plataformas de tipo Unix y GNU/Linux (Free Software Foundation, 2024a).

Tabla 8: Códigos de diagnóstico de error en la secuencia de arranque de LILO.

Salida en pantalla	Etapa alcanzada	Diagnóstico y causa raíz del fallo
(En blanco)	Ningún código cargado	El BIOS no encontró el MBR o no detectó la firma de arranque `0x55AA`.
`L`	Stage 1 ejecutado	Se cargó el primer sector, pero no se pudo localizar o leer el Stage 2 (error de disco).
`LI`	Stage 2 preliminar	El Stage 2 se cargó parcialmente, pero falló la correspondencia de direcciones geométricas.
`LIL`	Stage 2 en ejecución	Se leyó el Stage 2, pero la tabla de descriptores o el archivo de mapa `/boot/map` es ilegible.
`LIL-`	Descriptor corrupto	La tabla de descriptores del mapa físico contiene datos inconsistentes o dañados.
`LILO`	Inicialización completa	Todas las partes de LILO se cargaron con éxito en memoria; el sistema procede a arrancar.

Su diseño superó estructuralmente el paradigma de mapas de bloques físicos de LILO al incorporar controladores de sistemas de archivos nativos y un intérprete dinámico de comandos en tiempo de prearranque.

6.1. Evolución y arquitectura modular: de GRUB Legacy a GRUB 2

La evolución histórica del proyecto comprende dos generaciones arquitectónicamente divergentes:

1. **GRUB Legacy (versiones 0.9x):** Diseñado bajo una estructura monolítica, integraba sus funciones en una imagen estática principal (Stage 2). Aunque introdujo soporte para explorar directorios y una línea de órdenes interactiva, padecía severas restricciones de memoria en modo real x86, carecía de mecanismos de internacionalización y resultaba complejo de adaptar a arquitecturas no x86 o esquemas modernos de particionamiento. Su desarrollo activo cesó formalmente en 2005.
2. **GRUB 2 (versiones 1.90 en adelante):** Representó una reescritura integral del gestor desde sus cimientos, adoptando un diseño modular y estructurado en lenguaje C (Free Software Foundation, 2024a). En GRUB 2, las funcionalidades no esenciales residen en módulos dinámicos compilados con extensión `.mod` (equivalentes conceptualmente a controladores o módulos del kernel). El cargador incorpora un lenguaje de *scripting* completo con variables, estructuras condicionales y bucles, soporte de tipografías dinámicas y capacidad multiplataforma para arrancar sistemas en arquitecturas x86, x86-64, ARM, AArch64, MIPS, RISC-V y PowerPC sobre interfaces BIOS o UEFI.

6.2. Reconocimiento dinámico de sistemas de archivos

El avance conceptual determinante de GRUB sobre los cargadores de sectores estáticos radica en su pila de controladores de lectura de sistemas de archivos integrada en tiempo de ejecución. GRUB contiene módulos especializados para descifrar de manera autónoma estructuras como ext2, ext3, ext4, Btrfs, XFS, JFS, ZFS, FAT16, FAT32, NTFS, exFAT e ISO9660 (Free Software Foundation, 2024a).

Al encender la computadora, GRUB no lee listas de bloques arbitrarias, sino que monta lógicamente la partición de almacenamiento, decodifica los mapas de inodos o árboles B del sistema de archivos y localiza los binarios del kernel y del disco RAM inicial (*initial RAM disk*, initramfs) navegando directamente por la jerarquía de directorios (por ejemplo, `(hd0,gpt2)/boot/vmlinuz-linux`). Esta independencia física aporta beneficios operacionales inmediatos:

- **Invarianza ante actualizaciones:** Si se actualiza el núcleo del sistema operativo, se reemplaza la imagen del initramfs o el sistema de archivos experimenta fragmentación o reubicación de bloques lógicos, no se requiere reinstalar el cargador en el MBR ni sincronizar mapas en disco. GRUB resolverá la ruta nominal actualizada en el siguiente arranque sin intervención manual.
- **Soporte de abstracciones lógicas avanzadas:** Al cargar módulos dinámicos adicionales mediante la primitiva `insmod`, GRUB puede interpretar esquemas complejos de almacenamiento previo al inicio del kernel, tales como volúmenes del Administrador de Volúmenes Lógicos (*Logical Volume Manager*, LVM), matrices RAID por software (*Multiple Devices*, mdraid) y particiones cifradas con LUKS (*Linux Unified Key Setup*).

6.3. Proceso de carga, entorno de rescate y configuración declarativa

En sistemas x86 iniciados bajo firmware BIOS convencional, GRUB 2 divide su ejecución en componentes modulares diseñados para superar las restricciones de espacio físico, como se sintetiza en la Tabla 9 (Free Software Foundation, 2024a).

En equipos modernos con firmware UEFI, se suprime por completo el uso de `boot.img` y el espacio intermedio del disco; el firmware ejecuta directamente la aplicación `grubx64.efi` depositada en la partición ESP, transfiriendo el procesador en modo nativo de 64 bits a la rutina de inicialización de GRUB (Unified Extensible Firmware Interface Forum, 2024).

6.3.1. Consola interactiva y recuperación en fallos (grub rescue)

Si durante la fase de carga el componente `core.img` no logra montar la partición donde reside `/boot` (debido a alteraciones en la tabla de particiones, corrupción del sistema de archivos o inconsistencias de configuración), GRUB aborta la interfaz visual y activa la consola de rescate (`grub rescue>`) (Free Software Foundation, 2024a). Este entorno de depuración en modo texto provee comandos esenciales mínimos (`ls`, `set`, `insmod`) que permiten al administrador inspeccionar las unidades físicas conectadas, reasignar manualmente las variables de ruta (`root` y `prefix`), cargar el módulo funcional normal y recuperar el arranque del sistema operativo sin necesidad de recurrir a medios externos de rescate.

Tabla 9: Componentes medulares y etapas de ejecución de GRUB 2 en plataformas BIOS y UEFI.

Componente	Ubicación de almacenamiento	Función técnica en la secuencia de inicialización
`boot.img`	Sector MBR o VBR (512 bytes en LBA 0)	Código máquina de 16 bits; localiza y carga en memoria el primer sector del archivo `core.img`.
`core.img`	Espacio post-MBR (MBR gap) o partición BIOS Boot	Imagen sintetizada que contiene el núcleo de GRUB, soporte de particiones y el controlador básico del sistema de archivos de `/boot`.
Módulos `.mod`	Directorio `/boot/grub/i386-pc/`	Controladores dinámicos cargados en memoria bajo demanda mediante la primitiva `insmod` (RAID, LVM, cifrado, tipografías).
`grub.cfg`	Directorio `/boot/grub/`	Archivo de órdenes que define el menú visual, las entradas de sistemas operativos y los parámetros del kernel.
`grubx64.efi`	Partición ESP (`/EFI/<distro>/`)	Binario ejecutable en formato PE32+ para UEFI; agrupa el núcleo y los módulos esenciales en una sola imagen de 64 bits.

6.3.2. Autogeneración declarativa de configuración

En GRUB 2, el archivo maestro `/boot/grub/grub.cfg` no debe modificarse manualmente, pues constituye un archivo binario o guion compilado generado por herramientas automatizadas de espacio de usuario. La administración se centraliza en el archivo declarativo de parámetros globales `/etc/default/grub` y en la colección de scripts ejecutables modulares almacenados en `/etc/grub.d/` (como `10_linux` para núcleos instalados y `30_os-prober` para detectar sistemas operativos adicionales). Al invocar la utilidad `grub-mkconfig -o /boot/grub/grub.cfg` (o `update-grub` en distribuciones derivadas de Debian), el sistema compila estas directivas y sintetiza el archivo definitivo de arranque, garantizando consistencia operacional ante cualquier cambio en la topología de almacenamiento (Free Software Foundation, 2024a).

7. Discos básicos y discos dinámicos

En los sistemas operativos de la familia Microsoft Windows, el subsistema de almacenamiento clasifica las unidades de almacenamiento secundario en dos modelos arquitectónicos excluyentes: discos básicos y discos dinámicos (Microsoft Corporation, 2024a). Dicha categorización determina cómo el sistema operativo abstrae las regiones físicas de los discos magnéticos o unidades de estado sólido, cómo organiza las estructuras de metadatos de direccionamiento y el grado de flexibilidad disponible para redimensionar y enlazar los volúmenes en tiempo de ejecución.

7.1. Concepto y arquitectura del Administrador de Discos Lógicos

El esquema de **disco básico** constituye el modelo de almacenamiento estándar y tradicional de la industria. Un disco básico organiza su espacio mediante tablas de particiones fijas convencionales (MBR o GPT estándar) almacenadas en sectores dedicados al inicio o final del soporte físico (Silberschatz et al., 2006, §12.5.1, p. 418). Cada partición en un disco básico consiste estrictamente en una secuencia lineal e ininterrumpida de sectores contiguos confinada a una única unidad física. La tabla de particiones del disco básico es de visibilidad global y resulta legible de manera nativa por cualquier sistema operativo (DOS, Windows, Linux, macOS o firmware UEFI).

Por el contrario, el esquema de **disco dinámico**, introducido por Microsoft en Windows 2000 a partir de tecnología desarrollada con Veritas Software, desacopla la organización lógica de las restricciones geométricas del hardware mediante el Administrador de Discos Lógicos (*Logical Disk Manager*, LDM) (Microsoft Corporation, 2024a). En un disco dinámico, el sistema operativo prescinde de las particiones primarias y extendidas estándar; en su lugar, todo el disco físico se configura como un contenedor gestionado por una base de datos relacional oculta de metadatos de 1 MB de extensión.

Esta base de datos del LDM se aloja en el último megabyte del disco en medios MBR (o en una partición de metadatos de tipo GUID `5808C8AA-7E8F-42E0-85D7-E98C6970CD76` en discos GPT). La base de datos LDM almacena registros transaccionales detallados sobre las extensiones físicas de cada volumen, sus niveles de redundancia, identificadores únicos de grupo y esquemas de correspondencia de bloques. Gracias a esta arquitectura de base de datos distribuida, un conjunto de discos dinámicos comparte información de topología, lo que permite al núcleo del sistema operativo construir volúmenes complejos integrados por fragmentos inconexos o distribuidos a lo largo de múltiples discos físicos.

7.2. Tipología y modos de operación de volúmenes dinámicos

Los discos dinámicos reemplazan la noción de partición por el concepto de **volumen dinámico**, admitiendo cinco configuraciones operativas (Microsoft Corporation, 2024a; Stallings, 2018): el *volumen simple* (*simple volume*), confinado a un único disco en una o más extensiones no contiguas; el *volumen distribuido* (*spanned volume*), que concatena secuencialmente bloques libres de 2 a 32 discos físicos sin redundancia; el *volumen seccionado* (*striped volume*, RAID 0), que intercala franjas uniformes de 64 KB entre 2 y 32 unidades para maximizar el ancho de banda sin tolerancia a fallos; el *volumen reflejado* (*mirrored volume*, RAID 1), que duplica datos en tiempo real entre dos discos para alta disponibilidad; y el *volumen RAID-5*, que entrelaza datos y paridad por disyunción exclusiva (*exclusive OR*, XOR) a lo largo de un mínimo de tres discos físicos.

7.3. Diferencias estructurales y comparativa técnica

La distinción entre discos básicos y dinámicos involucra profundas diferencias de diseño en el soporte de subsistemas de almacenamiento, como se sintetiza en la Tabla 10 (Microsoft Corporation, 2024a; Stallings, 2018).

Tabla 10: Comparativa técnica integral entre discos básicos y discos dinámicos en Windows.

Criterio técnico	Disco básico	Disco dinámico (LDM)
Modelo de metadatos	Tablas estándar en sectores fijos (MBR en LBA 0 o matriz GPT primaria y secundaria).	Base de datos transaccional de metadatos LDM de 1 MB al final del disco o partición dedicada.
Contigüidad espacial	Estricta: las particiones son secuencias contiguas e ininterrumpidas de bloques en un disco.	Flexible: los volúmenes pueden formarse con múltiples extensiones inconexas de sectores.
Extensión multidisco	Inadmisible: una partición básica jamás puede abarcar más de una unidad física de disco.	Nativa: permite enlazar o intercalar capacidad de hasta 32 unidades físicas de almacenamiento.
Niveles RAID software	Ninguno: solo gestiona volúmenes simples independientes delimitados por particiones.	Soporte nativo por software para RAID 0 (seccionado), RAID 1 (reflejo) y RAID 5.
Modificación en caliente	Limitada: la extensión requiere espacio libre adyacente inmediato a la derecha de la partición.	Dinámica: permite expandir volúmenes incorporando extensiones no contiguas de cualquier disco del grupo.
Compatibilidad de SO	Universal: interoperable con DOS, Linux, macOS, Unix, Windows y firmware UEFI.	Restringida: formato propietario de Microsoft; incompatible con arranque múltiple y Linux nativo.

8. Creación y conversión de discos dinámicos

La transición desde el modelo tradicional de particiones fijas hacia la arquitectura de volúmenes dinámicos en entornos Windows no requiere el formateo previo del soporte físico ni el reordenamiento manual de los datos residentes. El sistema operativo provee mecanismos nativos tanto en interfaz gráfica como mediante consola interactiva para inicializar discos dinámicos o transformar unidades básicas existentes (Microsoft Corporation, 2024a).

8.1. Procedimientos administrativos de inicialización y conversión

El aprovisionamiento de discos dinámicos se gestiona mediante dos interfaces administrativas principales (Microsoft Corporation, 2024a, 2024b). En entornos interactivos de escritorio, la consola gráfica de Administración de Discos (`diskmgmt.msc`) permite localizar el nodo del disco físico, desplegar el menú contextual y ordenar la conversión hacia un grupo de almacenamiento (*disk group*); las particiones primarias y unidades lógicas preexistentes se transforman automáticamente en volúmenes dinámicos simples sin pérdida de datos. Por su parte, en despliegues automatizados o sesiones de recuperación de bajo nivel, la utilidad interactiva de línea de comandos `diskpart` provee un control granular exhaustivo mediante la selección de la unidad física de destino (`select disk <n>`), la ejecución de la transformación estructural (`convert dynamic`) y el aprovisionamiento de volúmenes especializados sobre el espacio no asignado (`create volume simple`, `create volume stripe` para matrices RAID 0, o `create volume raid` para arreglos RAID 5 con paridad).

8.2. El requisito de reserva del último megabyte (base de datos LDM)

La condición física indispensable para que el Administrador de Discos Lógicos pueda inicializar un medio reside en la disponibilidad de al menos 1 MB (exactamente 2048 sectores de 512 bytes) de espacio no asignado al final de la unidad física de almacenamiento (Microsoft Corporation, 2024a).

En discos configurados bajo el esquema MBR, este último megabyte no debe estar cubierto por ninguna partición primaria ni extendida. Durante la conversión, el subsistema LDM toma posesión exclusiva de esta franja terminal para depositar la base de datos de metadatos del grupo de discos, la cual contiene las tablas de réplica transaccionales, las firmas GUID del volumen y la asignación de bloques virtuales a sectores físicos. Si un disco básico fue particionado por herramientas heredadas de terceros que ocuparon el medio hasta el último sector absoluto direccionable ($LBA = N - 1$), el comando de conversión aborta invariablemente, reportando un error de espacio insuficiente en disco debido a la imposibilidad de incrustar la base de datos LDM sin truncar los sistemas de archivos existentes.

En discos particionados mediante GPT, el estándar de firmware UEFI reserva explícitamente particiones dedicadas identificadas por GUID: la partición de metadatos LDM (`5808C8AA-7E8F-42E0-85D7-E98C6970CD76`) y la partición de datos LDM (`AF9B60A0-1431-4F62-BC68-3311714A69AD`), preservando la integridad de las estructuras sin recurrir a sectores no declarados al margen de la tabla.

8.3. Asimetría de reversibilidad y consideraciones operativas

El ciclo de vida de un disco dinámico exhibe una marcada asimetría operativa: mientras que la migración de básico a dinámico se ejecuta en caliente y de manera no destructiva, la operación inversa hacia disco básico constituye un procedimiento estrictamente destructivo (Microsoft Corporation, 2024a).

El Administrador de Discos y la orden `convert basic` de `diskpart` prohíben la reversión si el disco dinámico alberga algún volumen activo. Para devolver el soporte físico al estado básico, el administrador está forzado a eliminar secuencialmente todos los volúmenes simples, distribuidos o redundantes alojados en la unidad, ocasionando la pérdida total e irreversible de los sistemas de archivos y de los datos contenidos si no se respalda previamente la información en un medio externo. Únicamente cuando la base de datos LDM queda vacía y el medio vuelve a constar enteramente de espacio no asignado, el sistema operativo permite reinicializar el disco bajo una tabla de particiones clásica MBR o GPT.

9. Ventajas, limitaciones y estado tecnológico

La introducción de los discos dinámicos en el ecosistema Windows representó un avance sustancial frente a la rigidez de las particiones clásicas, pero su diseño propietario impuso severas restricciones arquitectónicas que determinaron su posterior sustitución por modelos contemporáneos de virtualización de almacenamiento (Microsoft Corporation, 2024a).

9.1. Ventajas operativas y de resiliencia

Entre los beneficios técnicos más destacados de los discos dinámicos sobresale la gestión y redimensionamiento en caliente, permitiendo expandir volúmenes simples o distribui-

dos en producción sin reiniciar el sistema operativo ni desmontar servicios. Asimismo, habilita la agregación multidisco sin controladora dedicada para consolidar hasta 32 unidades físicas en volúmenes distribuidos (*spanned*) o seccionados (*striped* RAID 0), paralelizando las transferencias sobre múltiples canales. Por último, provee esquemas integrados de tolerancia a fallos por software (RAID 1 por reflejo y RAID 5 por paridad distribuida) que garantizan disponibilidad ante averías mecánicas sin tarjetas RAID propietarias (Silberschatz et al., 2006, §12.7, pp. 426–428).

9.2. Desventajas, riesgos operativos e incompatibilidad

A pesar de sus virtudes operativas, el esquema exhibe limitaciones estructurales críticas: impone incompatibilidad absoluta en entornos multiarranque (*multiboot*), pues los metadatos LDM son propietarios de Microsoft y resultan ilegibles para Linux (que requiere controladores experimentales de ingeniería inversa como `ldmtool`); prohíbe taxativamente la conversión de medios extraíbles (unidades USB o tarjetas SD); y expone a fallos catastróficos a los volúmenes distribuidos y seccionados, donde la avería de un solo disco corrompe todo el arreglo. Además, al trasladar unidades entre equipos, Windows las cataloga como discos dinámicos foráneos (*foreign disks*), exigiendo una importación manual de metadatos propensa a inconsistencias si el grupo está incompleto (Microsoft Corporation, 2024a).

9.3. Obsolescencia y sustitución por Espacios de Almacenamiento (Storage Spaces)

A partir de Windows 10 (compilación 1709) y Windows Server 2012, Microsoft declaró formalmente obsoletos los discos dinámicos, desaconsejando su adopción en nuevos despliegues y congelando su mantenimiento (Microsoft Corporation, 2024a).

La obsolescencia del LDM dio paso a **Espacios de Almacenamiento** (*Storage Spaces*), una capa de abstracción y virtualización de almacenamiento a nivel de kernel introducida en Windows Server 2012 (Microsoft Corporation, 2024c). A diferencia de los discos dinámicos, Espacios de Almacenamiento opera sobre discos básicos convencionales MBR o GPT agrupados en un fondo común de almacenamiento (*storage pool*). Sobre este grupo se provisionan discos virtuales con resiliencia avanzada (reflejo bidireccional y tridireccional, paridad dual tipo RAID 6), aprovisionamiento fino (*thin provisioning*), niveles jerarquizados de rendimiento (*storage tiering* combinando unidades de estado sólido y discos magnéticos) y soporte integral del sistema de archivos resiliente ReFS (*Resilient File System*), erradicando definitivamente las limitaciones de la base de datos LDM heredada (Microsoft Corporation, 2024c).

10. Conclusiones personales

El análisis sistemático del particionamiento, formateo y arranque de medios de almacenamiento secundario evidencia cómo la evolución de los sistemas operativos ha estado intrínsecamente ligada a la necesidad de desacoplar las estructuras lógicas de las limitaciones electromecánicas impuestas por el hardware físico. La superación del esquema de direccionamiento geométrico cilindro-cabeza-sector (CHS) en favor del direccionamiento por bloques lógicos (LBA) no constituyó un mero ajuste cuantitativo en la capacidad de

direccionamiento, sino una redefinición conceptual profunda: el hardware dejó de exponer una topología física arbitraria y rígida para proveer una matriz lineal homogénea de sectores de tamaño uniforme, delegando en microcontroladores integrados la detección y sustitución dinámica de bloques defectuosos mediante listas de fábrica y listas crecientes de sectores de reserva, lo que permitió a los sistemas de archivos concentrarse en la asignación eficiente del espacio mediante unidades de asignación lógicas sin verse comprometidos por irregularidades mecánicas locales.

En esta misma línea de desarrollo histórico, el agotamiento del registro de arranque maestro (MBR) ilustra con claridad las consecuencias de concebir arquitecturas de almacenamiento a partir de supuestos dimensionales de corto plazo. Confinar la tabla de particiones fundamental a un único sector absoluto de 512 bytes en el primer bloque direccionable del disco, restringir la tabla a cuatro entradas primarias de 16 bytes y limitar las variables de conteo de sectores a 32 bits impuso no solo la barrera matemática insalvable de los 2.2 TB con sectores clásicos, sino un riesgo operacional permanente derivado de la carencia total de redundancia ante corrupción magnética en el bloque inicial. Por el contrario, el estándar de Tabla de Particiones GUID (GPT) articulado bajo la especificación UEFI representa un diseño de ingeniería maduro y robusto: al expandir el direccionamiento a 64 bits habilita capacidades volumétricas que cubren holgadamente las necesidades previsibles de la computación masiva, al tiempo que incorpora mecanismos esenciales de tolerancia a fallos, tales como la réplica simétrica de cabeceras y descriptores en ambos extremos físicos del medio, la verificación algorítmica de integridad mediante sumas de verificación cíclica CRC32 y la eliminación de colisiones identificatorias mediante identificadores globales únicos generados bajo estándares criptográficos y formales.

Por su parte, el estudio de los gestores de arranque revela una transición paradigmática equivalente en la fase de inicialización del computador, transitando desde cargadores pasivos dependientes de hardware hacia subsistemas modulares autónomos y dinámicos. El contraste entre LILO y GRUB subraya las limitaciones de un diseño desacoplado del sistema de archivos: al concebir al cargador primario como un mecanismo estático que transfería sectores apoyándose ciegamente en mapas de bloques físicos precalculados por utilidades en espacio de usuario, LILO quedaba atado a una fragilidad extrema donde cualquier modificación del núcleo o reubicación de sectores demandaba la reinstalación manual del mapa, bajo riesgo de dejar el equipo en un estado irrecuperable. La irrupción de GRUB y su consolidación en GRUB 2 transformó este panorama al integrar controladores embebidos capaces de interpretar directamente sistemas de archivos complejos en tiempo de ejecución, navegar dinámicamente por la jerarquía de directorios, interactuar con abstracciones de volúmenes lógicos, matrices RAID por software y contenedores cifrados, e interactuar con el firmware moderno UEFI mediante protocolos estandarizados. Esta capacidad de análisis lógico en el arranque dotó al administrador de un entorno interactivo y resiliente que convierte al gestor de arranque en un verdadero microentorno de diagnóstico y contingencia antes de la cesión definitiva del control al sistema operativo.

Asimismo, la trayectoria tecnológica de los discos dinámicos de Microsoft y el Administrador de Discos Lógicos (LDM) aporta una valiosa lección sobre los compromisos entre flexibilidad operativa e interoperabilidad arquitectónica. Si bien la introducción de volúmenes dinámicos representó un salto significativo frente a la inflexibilidad de las particiones contiguas del modelo básico al posibilitar la agregación de múltiples discos físicos, el redimensionamiento dinámico en caliente y la provisión de arreglos redundantes por software, su dependencia de una base de datos propietaria oculta al final del disco aisló a estos esquemas del resto del ecosistema computacional, generando incompatibilidad insalvable

en configuraciones multiarranque y condicionando su ciclo de vida a un procedimiento de reversión estrictamente destructivo. La eventual obsolescencia del LDM y su sustitución formal por Espacios de Almacenamiento en entornos Windows contemporáneos confirma que la gestión de almacenamiento moderno no debe fundamentarse en parches de metadatos sobre esquemas heredados, sino en plataformas de virtualización nativas sustentadas en fondos comunes de almacenamiento, aprovisionamiento elástico y redundancia desacoplada de la estructura física del disco.

En conclusión, la estructuración del almacenamiento masivo y los mecanismos de arranque constituyen el basamento crítico que condiciona la seguridad, el rendimiento y la disponibilidad de cualquier arquitectura computacional contemporánea. Comprender con rigor formal y precisión matemática los fundamentos del particionamiento físico y lógico, la organización de los descriptores de partición, la sincronización entre firmware y cargadores ejecutables y las abstracciones del núcleo del sistema operativo no representa únicamente un dominio teórico fundamental de las ciencias de la computación, sino una competencia técnica indispensable para proyectar, administrar y asegurar infraestructuras de cómputo resilientes, capaces de preservar la integridad y accesibilidad de los datos en un entorno tecnológico caracterizado por el crecimiento exponencial de los volúmenes de información.

Referencias

Almesberger, W., & Coffman, J. (2000). *Lilo: Generic boot loader for linux (user's guide)* [Version 21.6]. Consultado el 1 de noviembre de 2026, desde https://almesberger.net/projects/lilo/

Carretero Pérez, J., García Carballeira, F., Miguel Anasagasti, P. d., & Pérez Costoya, F. (2001). *Sistemas operativos: una visión aplicada* (1.ª ed.). McGraw-Hill/Interamericana de España.

Free Software Foundation. (2024a). *Gnu grub manual 2.12*. Consultado el 1 de noviembre de 2026, desde https://www.gnu.org/software/grub/manual/grub/grub.html

Free Software Foundation. (2024b). *Multiboot specification version 0.6.96*. Consultado el 1 de noviembre de 2026, desde https://www.gnu.org/software/grub/manual/multiboot/multiboot.html

Microsoft Corporation. (2024a). *Basic and dynamic disks*. Consultado el 1 de noviembre de 2026, desde https://learn.microsoft.com/en-us/windows/win32/fileio/basic-and-dynamic-disks

Microsoft Corporation. (2024b). *Diskpart*. Consultado el 1 de noviembre de 2026, desde https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/diskpart

Microsoft Corporation. (2024c). *Storage spaces overview*. Consultado el 1 de noviembre de 2026, desde https://learn.microsoft.com/en-us/windows-server/storage/storage-spaces/overview

Silberschatz, A., Galvin, P. B., & Gagne, G. (2006). *Fundamentos de sistemas operativos* (7.ª ed.). McGraw-Hill/Interamericana de España.

Stallings, W. (2018). *Operating systems: Internals and design principles* (9.ª ed.) [Global Edition]. Pearson Education Limited.

Unified Extensible Firmware Interface Forum. (2024). *Unified extensible firmware interface (uefi) specification* (inf. téc. N.º Version 2.10). UEFI Forum. Consultado el 1 de noviembre de 2026, desde https://uefi.org/specifications